

CHAPTER 4: CONCURRENCY: SYNCHRONIZATION & DEADLOCK

Exercises:

ANSWERS

1. List four design issues for which the concept of concurrency is relevant. (Q 5.1)

Answer:

- Communication among processes,
- sharing of and competing for resources,
- synchronization of the activities of multiple processes,
- allocation of processor time to processes.

2. List the requirements for mutual exclusion. (Q 5.7)

Answer:

1. Mutual exclusion must be enforced: only one process at a time is allowed into its critical section, among all processes that have critical sections for the same resource or shared object.
2. A process that halts in its non-critical section must do so without interfering with other processes.
3. It must not be possible for a process requiring access to a critical section to be delayed indefinitely: no deadlock or starvation.
4. When no process is in a critical section, any process that requests entry to its critical section must be permitted to enter without delay.
5. No assumptions are made about relative process speeds or number of processors.
6. A process remains inside its critical section for a finite time only.

3. What operations can be performed on a semaphore? (Q 5.8)

Answer:

1. A semaphore may be initialized to a nonnegative value.
2. The *wait* operation decrements the semaphore value. If the value becomes negative, then the process executing the *wait* is blocked.
3. The *signal* operation increments the semaphore value. If the value is not positive, then a process blocked by a *wait* operation is unblocked.

4. What is a monitor? (Q 5.11)

Answer:

A monitor is a programming language construct providing abstract data types and mutually exclusive access to a set of procedures.

5. What is the distinction between *blocking* and *nonblocking* with respect to messages? (Q 5.12)

Answer:

There are two aspects, the send and receive primitives. When a *send* primitive is executed in a process, there are two possibilities: either the sending process is blocked until the message is received, or it is not. Similarly, when a process issues a *receive* primitive, there are two possibilities: If a message has previously been sent, the message is received and execution continues. If there is no waiting message, then either (a) the process is blocked until a message arrives, or (b) the process continues to execute, abandoning the attempt to receive.

6. Define race condition in the concurrency context. Give an example of a race condition.

Takrifkan suasana perlumbaan dalam konteks keserentakan. Berikan satu contoh suasana perlumbaan.

Answer:

A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.

Example: Consider a variable *x* shared by two processes, P1 and P2.

<u>P1 performs:</u>	<u>P2 performs:</u>
<i>x</i> = 1;	<i>x</i> = 2;
write <i>x</i> ;	write <i>x</i> ;

The value of *x* written (i.e. the final value of *x*) will depend on which process executes last, which will overwrite the earlier process' operation.

7. Define the following control problems which are associated with competing processes for resources:

Takrifkan masalah-masalah kawalan berikut yang dikaitkan dengan persaingan proses-proses terhadap sumber-sumber:

- i. Mutual exclusion
- ii. Deadlock
- iii. Starvation

- Mutual exclusion - the requirement that when one process is in a critical section that accesses shared resources, no other

process may be in a critical section that accesses any of those shared resources.

- Deadlock - a situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.
- Starvation - a situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen

8. What is the producer / consumer problem?

Apakah ia masalah pengeluar / pengguna?

- There are one or more producers generating some type of data (records, characters) and placing these in a buffer.
- There is a single consumer that is taking items out of the buffer one at a time.
- The system is to be constrained to prevent the overlap of buffer operations.
- That is, only one agent (producer or consumer) may access the buffer at any one time.

9. List the requirements for mutual exclusion.

Senaraikan keperluan-keperluan untuk 'mutual exclusion'.

- Only one process at a time is allowed in the critical section for a resource
- A process that halts in its noncritical section must do so without interfering with other processes
- No deadlock or starvation
- A process must not be delayed access to a critical section when there is no other process using it
- No assumptions are made about relative process speeds or number of processes
- A process remains inside its critical section for a finite time only

10. Concurrency arises in three different context. State these context.

(Persetujuan muncul dalam tiga konteks yang berbeza. Nyatakan semua konteks tersebut)

- Multiple applications
 - Multiprogramming
- Structured application
 - Application can be a set of concurrent processes
- Operating-system structure
 - Operating system is a set of processes or threads

11. Interrupt disabling is one of the hardware support approaches used to satisfy mutual exclusion requirements. However, this approach contributes into two major difficulties. State the difficulties of “interrupt disabling” approach.

“Interrupt disabling” adalah salah satu daripada pendekatan perkakasan sokongan yang digunakan untuk memenuhi keperluan “mutual exclusion”. Akan tetapi, pendekatan ini menyumbang kepada dua kerumitan. Nyatakan kerumitan pendekatan “interrupt disabling”.

- Processor is limited in its ability to interleave programs
- Cannot support into multiprocessing environment (disabling interrupts on one processor will not guarantee mutual exclusion)

12. What is semaphore? Explain how it is used in synchronizing concurrent processes.

Apakah itu semafor? Terangkan bagaimana ia digunakan dalam pensinkronian proses serentak.

Answer:

- Semaphore is a shared variable that has an integer value
- Special variable called a semaphore is used for signaling
- If a process is waiting for a signal, it is suspended until that signal is sent
- Use - to protect particular critical region or other resources.

13. A solution to the critical section problem must satisfy three requirements:

Satu penyelesaian kepada masalah seksyen genting mestilah memenuhi tiga syarat:

- Mutual exclusion (*pengasingan atas persetujuan*)
- Progress (*kemajuan*)
- Bounded waiting (*penantian terhad*)

Explain these requirements. (Terangkan syarat-syarat ini.)

- **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
- **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely

- **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
 - Assume that each process executes at a nonzero speed
 - No assumption concerning relative speed of the N processes

14. Define the following terms related to concurrency:

Takrifkan istilah-istilah berikut yang berkaitan dengan keserentakan:

- iv. Critical section (*bahagian kritikal*)
- v. Mutual exclusion (*pengasingan atas persetujuan*)
- vi. Race condition (*keadaan perlumbaan*)

- Critical section
 - A section of code within a process that requires access to shared resources and which may not be executed while another process is in a corresponding section of code.
- Mutual exclusion
 - The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
- Race condition
 - A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.